Module 1 (Foundations) — Post-Lecture Coding Questions

Instructions

- Use **Python 3.10+**. You may use `random`, `math`, and `statistics`.
- Keep your code readable (functions, docstrings, comments).
- For questions involving randomness, run multiple episodes and report summary statistics.
- You may submit a single `.py` file or a `.ipynb` notebook.

Shared Toy MDP (used in all questions)

We use an episodic, deterministic Markov Decision Process with:

- States: $\{A, B, T\}$ where T is terminal.
- Actions: $\{0, 1\}$.
- Discount factor: $\gamma = 0.9$.

Dynamics and rewards

State	Action	Next state	Reward
A	0	B	$+1$
A	1	T	0
B	0	A	0
B	1	T	$+2$

Stochastic policy π (used in Q1–Q2)

- At A: $\pi(0|A) = 0.7$, $\pi(1|A) = 0.3$.
- At B: $\pi(0|B) = 0.4$, $\pi(1|B) = 0.6$.

Discounted return

For an episode that generates rewards $r_1, r_2, \ldots, r_T$, the discounted return from time $t = 0$ is:

$$G_0 = \sum_{k=0}^{T-1} \gamma^k r_{k+1}.$$

1 Question 1 — Simulate episodes and compute returns (Monte Carlo)

Goal: Practice generating trajectory data and computing discounted returns.

Tasks

1. Implement the environment step function:

$$(s_{t+1}, r_{t+1}, \text{done}) = \texttt{step}(s_t, a_t).$$

2. Implement the given stochastic policy $\pi(a|s)$ and sample actions from it.
3. Simulate N episodes starting from $s_0 = A$.
4. Compute G_0 for each episode and report:
   - the mean return,
   - the standard deviation of returns.

Hints

- Store each transition as a tuple $(s, a, r, s', \text{done})$ (useful later).
- The reward is received **after** taking an action.
- Discounted return in Python:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t r_{t+1}.$$

Starter code (optional)

```
import random
from statistics import mean, pstdev

gamma = 0.9

def step(state, action):
    """Return (next_state, reward, done)."""
    if state == "A":
        if action == 0:
            return "B", 1.0, False
        else:
            return "T", 0.0, True
    if state == "B":
        if action == 0:
            return "A", 0.0, False
        else:
            return "T", 2.0, True
    return "T", 0.0, True

def policy_probs(state):
    """Return dict: action -> prob."""
    if state == "A":
        return {0: 0.7, 1: 0.3}
    if state == "B":
```

```
        return {0: 0.4, 1: 0.6}
    return {0: 1.0}

def sample_action(probs):
    x = random.random()
    cum = 0.0
    for a, p in probs.items():
        cum += p
        if x <= cum:
            return a
    return a  # fallback

def run_episode(max_steps=100):
    s = "A"
    rewards = []
    traj = []
    for _ in range(max_steps):
        a = sample_action(policy_probs(s))
        s_next, r, done = step(s, a)
        traj.append((s, a, r, s_next, done))
        rewards.append(r)
        s = s_next
        if done:
            break
    G = sum((gamma ** t) * r for t, r in enumerate(rewards))
    return G, traj

def main():
    N = 2000
    returns = []
    for _ in range(N):
        G, _ = run_episode()
        returns.append(G)
    print("Mean return:", mean(returns))
    print("Std dev (population):", pstdev(returns))

if __name__ == "__main__":
    main()
```

2 Question 2 — Policy evaluation via the Bellman expectation equations

Goal: Understand state-value functions and Bellman expectation equations.

Tasks

For the same MDP and the same stochastic policy π:

1. Compute $V^\pi(A)$ and $V^\pi(B)$ by solving the Bellman expectation equations.
2. Verify your results by comparing $V^\pi(A)$ with the Monte Carlo estimate from Question 1.

Bellman expectation equation

For any state s:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)\Big(R(s,a,s') + \gamma V^{\pi}(s')\Big).$$

Because the transitions here are deterministic, there is only one next state s' for each (s,a).

Hints

- You get two unknowns $V^{\pi}(A)$ and $V^{\pi}(B)$ (with $V^{\pi}(T) = 0$).
- Two solution approaches:
  1. **Algebra:** write two linear equations and solve.
  2. **Iterative policy evaluation:** repeatedly apply the Bellman expectation backup until convergence.
- Convergence criterion: stop when the maximum change Δ is below a tolerance (e.g., 10^{-10}).

Starter code (iterative policy evaluation)

```
gamma = 0.9

def transition_reward(state, action):
    if state == "A":
        return ("B", 1.0) if action == 0 else ("T", 0.0)
    if state == "B":
        return ("A", 0.0) if action == 0 else ("T", 2.0)
    return ("T", 0.0)

def pi(state):
    if state == "A":
        return {0: 0.7, 1: 0.3}
    if state == "B":
        return {0: 0.4, 1: 0.6}
    return {}

def policy_evaluation(tol=1e-10, max_iters=10000):
    V = {"A": 0.0, "B": 0.0, "T": 0.0}
    for _ in range(max_iters):
        delta = 0.0
        for s in ["A", "B"]:
            v_old = V[s]
            v_new = 0.0
            for a, p in pi(s).items():
                s_next, r = transition_reward(s, a)
                v_new += p * (r + gamma * V[s_next])
            V[s] = v_new
            delta = max(delta, abs(v_new - v_old))
        if delta < tol:
            break
    return V

if __name__ == "__main__":
    V = policy_evaluation()
    print("V^pi(A) =", V["A"])
```

```
    print("V^pi(B) =", V["B"])
```

3 Question 3 — Bellman optimality backup + ε-greedy action selection

Goal: Connect the Bellman optimality equation to an algorithmic backup, and practice exploration.

Tasks

1. Implement a function that performs one **Bellman optimality backup**:

$$Q_{\text{new}}(s,a) \leftarrow r + \gamma \max_{a'} Q(s',a').$$

2. Implement ε-greedy action selection:
   - with probability ε: choose a random action,
   - otherwise: choose $\arg\max_a Q(s,a)$ (break ties randomly).
3. Initialize all $Q(s,a) = 0$. Perform K sweeps of backups over all (s,a).
4. After each sweep, print the greedy action in A and B, and the updated Q-values.

Hints

- Use a dictionary for the Q-table: `Q[(s,a)]`.
- For the terminal state T, define $\max_{a'} Q(T,a') = 0$.
- Prefer **synchronous** sweeps: compute `Q_new` from `Q` and then replace.
- For tie-breaking, collect all best actions and sample uniformly among them.

Starter code

```
import random

gamma = 0.9
states = ["A", "B"]
actions = [0, 1]

def step_det(state, action):
    if state == "A":
        return ("B", 1.0) if action == 0 else ("T", 0.0)
    if state == "B":
        return ("A", 0.0) if action == 0 else ("T", 2.0)
    return ("T", 0.0)

def max_q_next(Q, s_next):
    if s_next == "T":
        return 0.0
    return max(Q[(s_next, a)] for a in actions)

def bellman_opt_backup(Q, s, a):
```

```
    s_next, r = step_det(s, a)
    return r + gamma * max_q_next(Q, s_next)

def epsilon_greedy(Q, s, eps=0.1):
    if random.random() < eps:
        return random.choice(actions)
    # greedy with random tie-break
    vals = [Q[(s, a)] for a in actions]
    m = max(vals)
    best = [a for a in actions if Q[(s, a)] == m]
    return random.choice(best)

def greedy_action(Q, s):
    vals = [Q[(s, a)] for a in actions]
    m = max(vals)
    best = [a for a in actions if Q[(s, a)] == m]
    return random.choice(best)

def run_sweeps(K=6):
    Q = {(s, a): 0.0 for s in states for a in actions}
    for k in range(1, K + 1):
        Q_new = Q.copy()
        for s in states:
            for a in actions:
                Q_new[(s, a)] = bellman_opt_backup(Q, s, a)
        Q = Q_new
        print(f"Sweep {k}: greedy(A)={greedy_action(Q,'A')}, greedy(B)
            ={greedy_action(Q,'B')}")
        print("  Q(A,0),Q(A,1) =", Q[('A',0)], Q[('A',1)])
        print("  Q(B,0),Q(B,1) =", Q[('B',0)], Q[('B',1)])
    return Q

if __name__ == "__main__":
    run_sweeps()
```

What to submit

- Your Python code for Q1–Q3.
- Short answers (1–3 sentences each):
  1. What does γ do conceptually?
  2. Why do we need exploration?
  3. What is the difference between Bellman *expectation* vs *optimality* equations?